LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 5 RELASI ANTAR KELAS

NAMA : Jeremy Marcello Waani

NIM : 2311110003

KELAS : PS1DS-05

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. **Dasar Teori**

Relasi antar kelas menjelaskan hubungan antara objek yang bekerja sama dalam program berorientasi objek. Praktikum ini membahas asosiasi, agregasi, dan komposisi melalui contoh dosen dan mahasiswa, jurusan dan mahasiswa, serta komputer dan komponennya. Ketiga relasi kemudian diterapkan pada sistem pesanan cafe untuk memisahkan data harga, kumpulan item, dan proses pembayaran.

Diagram kelas menggambarkan nama kelas, atribut, method, dan hubungan antarkelas. Simbol + menyatakan akses public, - menyatakan private, dan # menyatakan protected. Dalam modul, asosiasi ditunjukkan dengan garis penghubung, agregasi dengan belah ketupat kosong pada sisi objek penampung, dan komposisi dengan belah ketupat terisi pada sisi pemilik komponen.

Asosiasi menggambarkan interaksi antarobjek. Pada contoh praktikum, satu objek menggunakan method objek lain yang diterima melalui parameter. Dosen memberikan nilai kepada Mahasiswa tanpa menyimpan Mahasiswa sebagai atribut. Pada tugas cafe, Kasir membaca Pesanan hanya selama prosesPembayaran() berjalan.

Agregasi adalah hubungan keseluruhan dan bagian dengan objek bagian yang dibuat secara independen. Objek penampung menyimpan referensi ke objek bagian. Mahasiswa dibuat sebelum dimasukkan ke Jurusan, sedangkan ItemPesanan dibuat di Main sebelum ditambahkan ke Pesanan. Satu objek bagian tetap dapat dirujuk dari luar penampung.

Komposisi menunjukkan kepemilikan bagian yang lebih kuat. Objek pemilik membuat komponen internalnya sendiri. Komputer membuat Processor, Monitor, dan Mouse melalui constructor; ItemPesanan membuat RincianHarga. Dalam Java, kepemilikan ini dinyatakan melalui rancangan class dan referensi objek. Memori objek baru dapat direklamasi ketika objek tersebut tidak lagi terjangkau.

Enkapsulasi mendukung relasi tersebut dengan menyembunyikan field melalui private dan menyediakan akses terkontrol melalui method. Constructor mengisi keadaan awal objek, sedangkan this membedakan field dari parameter dengan nama yang sama. Array bertipe tetap menyimpan referensi objek; penghitung jumlah item membatasi bagian array yang sudah terisi.

Relasi	Cara objek digunakan	Contoh pada cafe
Asosiasi	Diterima melalui parameter method	Kasir dengan Pesanan
Agregasi	Dibuat di luar lalu disimpan	Pesanan dengan ItemPesanan
Komposisi	Dibuat di constructor pemilik	ItemPesanan dengan RincianHarga

Sumber teori: MODUL-5.pdf, bagian 5.1 dan 5.2.1 sampai 5.2.3, halaman tercetak 24 sampai 30.

B. **Guided**

1. **Asosiasi antara Dosen dan Mahasiswa**

Contoh asosiasi ditempatkan pada package guided.asosiasi. Mahasiswa menyimpan NIM, nama, dan nilai. Dosen menggunakan objek Mahasiswa sebagai parameter saat memberikan atau membaca nilai. Program utama berada pada class Dosen.

guided/asosiasi/Mahasiswa.java

```
package guided.asosiasi;

public class Mahasiswa {
    private String nim;
    private String nama;
    private int nilai;

    public void setNama(String nama){
        this.nama = nama;
    }

    public void setNim(String nim){
        this.nim = nim;
    }

    public void setNilai(int nilai){
        this.nilai = nilai;
    }

    public int getNilai(){
        return this.nilai;
    }

    public String getNama(){
        return this.nama;
    }

    public String getNim(){
        return this.nim;
    }
}
```

guided/asosiasi/Dosen.java

```
package guided.asosiasi;

public class Dosen {
    private String kodeDosen;
    private String namaDosen;

    public void setKodeDosen(String kodeDosen){
        this.kodeDosen = kodeDosen;
    }

    public void setNamaDosen(String namaDosen){
        this.namaDosen = namaDosen;
    }
```

guided/asosiasi/Dosen.java

```
    public String getKodeDosen(){
        return this.kodeDosen;
    }

    public String getNamaDosen(){
        return this.namaDosen;
    }

    public void giveScore(Mahasiswa s, int nilai){
        s.setNilai(nilai);
    }

    public int getScore(Mahasiswa s){
        return s.getNilai();
    }

    public static void main(String[] args){
        Mahasiswa m = new Mahasiswa();

        m.setNim("130118383");
        m.setNama("Budi");

        Dosen d = new Dosen();
        d.giveScore(m, 90);

        System.out.println("Nim : " + m.getNim());
        System.out.println("Nama : " + m.getNama());
        System.out.println("Nilai : " + d.getScore(m));
    }

}
```

Output

```
java guided.asosiasi.Dosen
```

```
Nim : 130118383
Nama : Budi
Nilai : 90
```

Gambar 1 Output guided.asosiasi.Dosen

Penjelasan

Objek m diberi NIM 130118383 dan nama Budi melalui setter. Pemanggilan d.giveScore(m, 90) menjalankan setNilai(90) pada objek mahasiswa yang sama. Method getScore(m) kemudian membaca nilai tersebut. Nilai 90 pada output menunjukkan bahwa dua objek berinteraksi melalui method, sementara class Dosen tidak memiliki field bertipe Mahasiswa.

2. **Agregasi antara Jurusan dan Mahasiswa**

Package guided.agregasi berisi Mahasiswa, Jurusan, dan Main. Mahasiswa dibuat secara terpisah, lalu referensinya dimasukkan ke array daftarMahasiswa milik Jurusan. Pemisahan package membedakan class Mahasiswa pada contoh ini dari class dengan nama yang sama pada asosiasi.

guided/agregasi/Mahasiswa.java

```
package guided.agregasi;

public class Mahasiswa {
    private String NIM;
    private String Nama;

    public Mahasiswa (String NIM, String Nama){
        this.NIM = NIM;
        this.Nama = Nama;
    }

    public String getNim(){
        return (NIM) ;
    }

    public String getNama(){
        return (Nama) ;
    }

    public void setNim(String NIM){
        this.NIM = NIM;
    }

    public void setNama(String Nama){
        this.Nama = Nama;
    }

}
```

guided/agregasi/Jurusan.java

```
package guided.agregasi;

public class Jurusan {
    private static int jumlahMahasiswa;
    private String kodeJurusan, namaJurusan;
    private Mahasiswa[] daftarMahasiswa = new Mahasiswa[32];

    //constructor
    public Jurusan(String kodeJurusan, String namaJurusan){
        this.kodeJurusan = kodeJurusan;
        this.namaJurusan = namaJurusan;
    }

    //behavior
    public void addMahasiswa(Mahasiswa m){
        this.daftarMahasiswa[jumlahMahasiswa] = m;
        jumlahMahasiswa++;
    }
```

guided/agregasi/Jurusan.java

```
    public void displayMahasiswa(){
        System.out.println("Kode Jurusan: " + this.kodeJurusan);
        System.out.println("Nama Jurusan: " + this.namaJurusan);

        System.out.println("Daftar Mahasiswa: ");

        for (int i = 0; i<jumlahMahasiswa; i++){
            System.out.println(i+1+ ". " +
            daftarMahasiswa[i].getNim() + "  "
            + daftarMahasiswa[i].getNama());
        }
    }

    //getter
    public String getKodeJurusan(){
        return kodeJurusan;
    }

    public String getNamaJurusan(){
        return namaJurusan;
    }

    //setter
    public void setKodeJurusan(String kodeJurusan){
        this.kodeJurusan = kodeJurusan;
    }

    public void setNamaJurusan(String namaJurusan){
        this.namaJurusan = namaJurusan;
    }


}
```

guided/agregasi/Main.java

```
package guided.agregasi;

public class Main {
    public static void main(String[] args) {
        Mahasiswa m1 = new Mahasiswa("001", "Max");
        Mahasiswa m2 = new Mahasiswa("002", "Min");

        Jurusan prodi = new Jurusan("DS", "Data Science");
        prodi.addMahasiswa(m1);
        prodi.addMahasiswa(m2);
        prodi.displayMahasiswa();
    }
}
```

Output

```
java guided.agregasi.Main
```

```
Kode Jurusan: DS
Nama Jurusan: Data Science
Daftar Mahasiswa:
1. 001  Max
2. 002  Min
```

Gambar 2 Output guided.agregasi.Main

Penjelasan

Main membuat m1 dengan NIM 001 dan nama Max serta m2 dengan NIM 002 dan nama Min. Jurusan berkode DS dan bernama Data Science menerima kedua objek melalui addMahasiswa(). Method displayMahasiswa() menampilkan data jurusan dan mahasiswa sesuai urutan penambahan. Objek mahasiswa sudah ada sebelum objek jurusan menerima referensinya, sehingga relasinya merupakan agregasi.

Pada kode guided yang digunakan, jumlahMahasiswa bersifat static dan dipakai bersama oleh semua objek Jurusan. Contoh ini hanya membuat satu jurusan. Untuk penggunaan banyak jurusan, penghitung sebaiknya menjadi atribut instance agar jumlah setiap daftar terpisah. Array berkapasitas 32 juga belum disertai pemeriksaan kapasitas pada addMahasiswa().

3. **Komposisi Komputer dan Komponen**

Package guided.komposisi berisi Processor, Monitor, Mouse, Komputer, dan Main. Komputer memiliki tiga objek komponen sebagai field private. Seluruh komponen dibuat di dalam constructor Komputer menggunakan data yang diterima dari program utama.

guided/komposisi/Processor.java

```
package guided.komposisi;

public class Processor {
    private String brand;
    private int Kecepatan;

    public Processor(String brand, int kecepatan){
        this.brand = brand;
        this.Kecepatan = kecepatan;
    }

    //behavior
    public void displaySpec(){
        System.out.println(this.brand + " " + this.Kecepatan);

    }
}
```

guided/komposisi/Monitor.java

```
package guided.komposisi;

public class Monitor {
    private String brand;

    public Monitor(String brand){
        this.brand = brand;
    }

    public void DisplaySpec() {
        System.out.println(this.brand);
    }

}
```

guided/komposisi/Mouse.java

```
package guided.komposisi;

public class Mouse {
    private String brand, tipe;

    public Mouse(String brand, String tipe){
        this.brand = brand;
        this.tipe = tipe;
    }

    public void displaySpec(){
        System.out.println(this.brand + ", " + this.tipe);
    }

}
```

guided/komposisi/Komputer.java

```
package guided.komposisi;

public class Komputer {
    private Processor processor;
    private Monitor monitor;
    private Mouse mouse;

    public Komputer(String brandCpu, int kecepatanCpu, String brandMonitor,
String brandMouse, String tipeMouse){
        this.processor = new Processor(brandCpu, kecepatanCpu);
        this.monitor = new Monitor(brandMonitor);
        this.mouse = new Mouse(brandMouse, tipeMouse);
    }

    public void displaySpecKomputer() {
        System.out.println("=== SPESIFIKASI KOMPUTER ===");
        this.processor.displaySpec();
        this.monitor.DisplaySpec();
        this.mouse.displaySpec();
    }
}
```

guided/komposisi/Main.java

```
package guided.komposisi;

public class Main {
    public static void main(String[] args) {
        Komputer mac = new Komputer("Apple M2", 3200, "Apple Display",
"Logitech", "Wireless");
        mac.displaySpecKomputer();
    }
}
```

Output

```
java guided.komposisi.Main
```

```
=== SPESIFIKASI KOMPUTER ===
Apple M2 3200
Apple Display
Logitech, Wireless
```

Gambar 3 Output guided.komposisi.Main

Penjelasan

Pembuatan Komputer dengan Processor Apple M2 berkecepatan 3200, monitor Apple Display, dan mouse Logitech bertipe Wireless memicu pembuatan tiga objek di dalam constructor. Main cukup membuat satu objek Komputer. Pemanggilan displaySpecKomputer() meneruskan pencetakan ke setiap komponen dan menghasilkan spesifikasi sesuai data awal.

Program penggabung guided

Class guided.Main menyediakan runAgregasi() dan runKomposisi(). Kedua method memanggil program utama dari package masing-masing, sehingga kedua contoh juga dapat dijalankan secara berurutan. Contoh asosiasi tetap dijalankan melalui guided.asosiasi.Dosen.

guided/Main.java

```
package guided;

public class Main {

    public static void main(String[] args) {
        runAgregasi();
        runKomposisi();
    }

    public static void runAgregasi(){
        guided.agregasi.Main.main(new String[0]);
    }

    public static void runKomposisi(){
        guided.komposisi.Main.main(new String[0]);
    }


}
```

C. **Unguided**

1. **Sistem Manajemen Pesanan Cafe**

   Soal

Sebuah cafe memerlukan program Java untuk mencatat item pesanan dan memproses pembayaran. Program harus memuat class RincianHarga, ItemPesanan, Pesanan, Kasir, dan Main. Seluruh class dikelompokkan dalam package unguided.

Tugas komposisi: ItemPesanan mempunyai namaItem dan objek RincianHarga. RincianHarga menyimpan hargaDasar dan nominal pajak 10%, serta menyediakan getHargaTotal(). Objek rincian harus dibuat dengan new di constructor ItemPesanan, bukan dibuat langsung di Main.

Tugas agregasi: Pesanan menyimpan nomorMeja, array ItemPesanan[], dan jumlahItem. Item dibuat di Main sebelum dimasukkan melalui tambahItem(). Pesanan menyediakan tampilkanPesanan() dan getter untuk membaca data. Array harus mempunyai kapasitas tetap tanpa ArrayList atau koleksi lainnya.

Tugas asosiasi: Kasir menyimpan namaKasir dan memproses objek Pesanan melalui parameter prosesPembayaran(Pesanan pesanan). Kasir dilarang menyimpan Pesanan sebagai field. Proses pembayaran menghitung subtotal, pajak, dan total bayar untuk dicetak sebagai struk.

Tugas eksekusi: Main membuat kasir Budi, tiga item pada tabel berikut, dan Pesanan untuk Meja 4 dengan kapasitas 5 item. Item ditambahkan satu per satu, daftar pesanan ditampilkan, kemudian kasir memproses pembayaran. Semua field dienkapsulasi; nama class memakai PascalCase, variabel dan method memakai camelCase, serta constructor memakai this. Tidak diperlukan inheritance, overriding, atau abstract class.

Item	Harga dasar	Pajak 10%	Harga total
Iced Americano	Rp 18.000	Rp 1.800	Rp 19.800
Beef Croissant	Rp 25.000	Rp 2.500	Rp 27.500
Matcha Latte	Rp 22.000	Rp 2.200	Rp 24.200

Sumber soal: soal_unguided_modul_5.md, Tugas 1 sampai Tugas 4.

Jawaban

unguided/RincianHarga.java

```
package unguided;

public class RincianHarga {
    private final double hargaDasar;
    private final double pajak;

    public RincianHarga(double hargaDasar) {
        if (!Double.isFinite(hargaDasar) || hargaDasar < 0) {
            throw new IllegalArgumentException("Harga dasar harus berupa angka
yang tidak negatif.");
        }

        this.hargaDasar = hargaDasar;
        this.pajak = this.hargaDasar * 0.10;
    }

    public double getHargaDasar() {
        return this.hargaDasar;
    }

    public double getPajak() {
        return this.pajak;
    }

    public double getHargaTotal() {
        return this.hargaDasar + this.pajak;
    }
}
```

unguided/ItemPesanan.java

```
package unguided;

public class ItemPesanan {
    private final String namaItem;
    private final RincianHarga rincian;

    public ItemPesanan(String namaItem, double hargaDasar) {
        if (namaItem == null || namaItem.trim().isEmpty()) {
            throw new IllegalArgumentException("Nama item tidak boleh kosong.");
        }

        this.namaItem = namaItem;
        // Komposisi: rincian harga dibuat dan dimiliki oleh item pesanan ini.
        this.rincian = new RincianHarga(hargaDasar);
    }

    public String getNamaItem() {
        return this.namaItem;
    }

    public RincianHarga getRincian() {
        return this.rincian;
    }
}
```

unguided/Pesanan.java

```
package unguided;

public class Pesanan {
    private final int nomorMeja;
    private final ItemPesanan[] daftarItem;
    private int jumlahItem;

    public Pesanan(int nomorMeja, int kapasitas) {
        if (nomorMeja <= 0) {
            throw new IllegalArgumentException("Nomor meja harus lebih dari
nol.");
        }
        if (kapasitas <= 0) {
            throw new IllegalArgumentException("Kapasitas pesanan harus lebih
dari nol.");
        }

        this.nomorMeja = nomorMeja;
        this.daftarItem = new ItemPesanan[kapasitas];
        this.jumlahItem = 0;
    }

    public void tambahItem(ItemPesanan item) {
        if (item == null) {
            throw new IllegalArgumentException("Item pesanan tidak boleh null.");
        }
        if (this.jumlahItem >= this.daftarItem.length) {
            throw new IllegalStateException("Kapasitas pesanan sudah penuh.");
        }

        // Agregasi: simpan referensi item yang sudah dibuat di luar Pesanan.
        this.daftarItem[this.jumlahItem] = item;
        this.jumlahItem++;
    }

    public void tampilkanPesanan() {
        System.out.println("=== DAFTAR PESANAN CAFE ===");
        System.out.println("Nomor Meja: " + this.nomorMeja);

        for (int i = 0; i < this.jumlahItem; i++) {
            ItemPesanan item = this.daftarItem[i];
            RincianHarga rincian = item.getRincian();

            System.out.println((i + 1) + ". " + item.getNamaItem()
                    + " - Rp " + rincian.getHargaTotal()
                    + " (Harga: " + rincian.getHargaDasar()
                    + ", Pajak: " + rincian.getPajak() + ")");
        }
    }

    public int getNomorMeja() {
        return this.nomorMeja;
    }

    public ItemPesanan[] getDaftarItem() {
        // Salin array agar pemanggil tidak bisa mengganti isi array internal.
        // Referensi item tetap sama, sesuai hubungan agregasi.
        return this.daftarItem.clone();
    }
```

unguided/Pesanan.java

```
    public int getJumlahItem() {
        return this.jumlahItem;
    }
}
```

unguided/Kasir.java

```
package unguided;

public class Kasir {
    private final String namaKasir;

    public Kasir(String namaKasir) {
        if (namaKasir == null || namaKasir.trim().isEmpty()) {
            throw new IllegalArgumentException("Nama kasir tidak boleh kosong.");
        }

        this.namaKasir = namaKasir;
    }

    // Asosiasi: Pesanan hanya digunakan melalui parameter, bukan sebagai field.
    public void prosesPembayaran(Pesanan pesanan) {
        if (pesanan == null) {
            throw new IllegalArgumentException("Pesanan tidak boleh null.");
        }

        double totalSubtotal = 0;
        double totalPajak = 0;
        double totalBayar = 0;
        ItemPesanan[] daftarItem = pesanan.getDaftarItem();

        for (int i = 0; i < pesanan.getJumlahItem(); i++) {
            RincianHarga rincian = daftarItem[i].getRincian();
            totalSubtotal += rincian.getHargaDasar();
            totalPajak += rincian.getPajak();
            totalBayar += rincian.getHargaTotal();
        }

        System.out.println("=== STRUK PEMBAYARAN KASIR ===");
        System.out.println("Kasir Bertugas : " + this.namaKasir);
        System.out.println("Pesanan        : Meja " + pesanan.getNomorMeja());
        System.out.println("Jumlah Item    : " + pesanan.getJumlahItem());
        System.out.println("Total Subtotal : Rp " + totalSubtotal);
        System.out.println("Total Pajak    : Rp " + totalPajak);
        System.out.println("TOTAL BAYAR    : Rp " + totalBayar);
        System.out.println("==============================");
    }
}
```

unguided/Main.java

```
package unguided;

public class Main {
    public static void main(String[] args) {
        Kasir kasir = new Kasir("Budi");

        // Item dibuat secara independen sebelum dimasukkan ke dalam pesanan.
        ItemPesanan americano = new ItemPesanan("Iced Americano", 18000);
        ItemPesanan croissant = new ItemPesanan("Beef Croissant", 25000);
        ItemPesanan matcha = new ItemPesanan("Matcha Latte", 22000);

        Pesanan pesanan = new Pesanan(4, 5);
        pesanan.tambahItem(americano);
        pesanan.tambahItem(croissant);
        pesanan.tambahItem(matcha);

        pesanan.tampilkanPesanan();
        System.out.println();
        kasir.prosesPembayaran(pesanan);
    }
}
```

Output

java unguided.Main

```
=== DAFTAR PESANAN CAFE ===
Nomor Meja: 4
1. Iced Americano - Rp 19800.0 (Harga: 18000.0, Pajak: 1800.0)
2. Beef Croissant - Rp 27500.0 (Harga: 25000.0, Pajak: 2500.0)
3. Matcha Latte - Rp 24200.0 (Harga: 22000.0, Pajak: 2200.0)

=== STRUK PEMBAYARAN KASIR ===
Kasir Bertugas : Budi
Pesanan        : Meja 4
Jumlah Item    : 3
Total Subtotal : Rp 65000.0
Total Pajak    : Rp 6500.0
TOTAL BAYAR    : Rp 71500.0
==============================
```

Gambar 4 Output unguided.Main

Penjelasan relasi antar kelas

RincianHarga menghitung pajak sebagai hargaDasar dikalikan 0,10. getHargaTotal() menjumlahkan harga dasar dan pajak. Constructor menolak harga negatif, NaN, dan nilai tak hingga. Kedua field bersifat private final dan hanya dibaca melalui getter, sehingga harga yang sudah terbentuk tidak dapat diubah melalui antarmuka class.

ItemPesanan menyimpan nama item dan langsung membuat new RincianHarga(hargaDasar) pada constructor. Inilah bagian komposisi. Main tidak membuat RincianHarga secara langsung. getRincian() menyediakan akses baca ke objek rincian yang tidak memiliki setter.

Pesanan mengalokasikan array sesuai kapasitas dan menyimpan item eksternal melalui tambahItem(). jumlahItem mencatat posisi pengisian berikutnya sekaligus batas perulangan. Item null dan penambahan saat array penuh ditolak sebelum data berubah. jumlahItem menjadi atribut instance sehingga jumlah item dari pesanan berbeda tidak bercampur.

getDaftarItem() mengembalikan clone array. Mengganti elemen pada array hasil getter tidak mengubah susunan array internal Pesanan. Referensi ItemPesanan di dalam salinan tetap menunjuk objek yang sama; hal ini mempertahankan agregasi tanpa membuka akses langsung untuk mengganti data penampung.

Kasir hanya menerima Pesanan sebagai parameter. Method prosesPembayaran() membaca item melalui getter dan mengakumulasikan harga dasar, pajak, serta harga total dengan perulangan. Total disimpan sebagai variabel lokal, sehingga pembayaran pesanan berikutnya dihitung kembali dari nol.

Analisis hasil perhitungan

Ketiga harga dasar dijumlahkan menjadi 18.000 + 25.000 + 22.000 = 65.000. Total pajak adalah 1.800 + 2.500 + 2.200 = 6.500. Dengan demikian, total pembayaran adalah 65.000 + 6.500 = 71.500. Hasil ini juga sama dengan penjumlahan harga total item, yaitu 19.800 + 27.500 + 24.200. Output menggunakan akhiran .0 karena nilai harga disimpan sebagai double.

Komponen	Hasil
Nomor meja dan jumlah item	Meja 4 dengan 3 item
Subtotal seluruh item	Rp 65.000
Total pajak	Rp 6.500
Total pembayaran	Rp 71.500

Pengujian program

Build Maven verify berhasil menggunakan JDK 27 dengan target Java 26. Seluruh 16 file Java dapat dikompilasi, dan program utama pada setiap package dapat dijalankan. Output unguided dibandingkan dengan contoh pada soal dan cocok pada seluruh baris. Pemeriksaan perilaku tambahan dijalankan melalui program uji terpisah dengan hasil berikut.

Skenario uji	Hasil pengamatan
Perhitungan tiga item	Subtotal 65000.0, pajak 6500.0, total 71500.0.
Array terisi penuh atau menerima null	Penambahan ditolak; jumlah dan item lama tetap.
Perubahan array hasil getter	Susunan array internal pesanan tetap.
Dua pesanan menggunakan satu item	Referensi item sama; jumlah tiap pesanan terpisah.
Pembayaran ulang dan pesanan kosong	Total tidak terakumulasi ulang; pesanan kosong bernilai 0.0.
Harga dan identitas tidak valid	Harga negatif, NaN, tak hingga, dan nama kosong ditolak.
Enkapsulasi dan asosiasi	Semua field private; Kasir tidak memiliki field Pesanan.

D. Kesimpulan

Praktikum Modul 5 menunjukkan bahwa pemilihan relasi antar kelas ditentukan oleh cara objek dibuat, dimiliki, dan digunakan. Asosiasi memungkinkan interaksi melalui parameter method. Agregasi menyimpan referensi objek yang dibuat secara independen, sedangkan komposisi membentuk komponen di dalam objek pemiliknya.

Pada guided, relasi tersebut ditunjukkan oleh Dosen dan Mahasiswa, Jurusan dan Mahasiswa, serta Komputer dan komponennya. Pada unguided, ItemPesanan membentuk RincianHarga, Pesanan menampung ItemPesanan dengan array tetap, dan Kasir memproses Pesanan melalui parameter. Enkapsulasi menjaga agar data dibaca dan diubah melalui antarmuka yang ditentukan class.

Sistem cafe menghasilkan subtotal Rp 65.000, pajak Rp 6.500, dan total bayar Rp 71.500 untuk tiga item pada Meja 4. Hasil eksekusi sesuai contoh soal. Pengujian juga menunjukkan bahwa pemeriksaan kapasitas, salinan array pada getter, dan penghitung per objek membantu menjaga konsistensi data pesanan.

Sumber

Modul 5 Relasi antar Kelas. Berkas MODUL-5.pdf pada Praktikum/modul/guided, halaman tercetak 24 sampai 30.

Soal Unguided Modul 5 Sistem Manajemen Pesanan Cafe. Berkas soal_unguided_modul_5.md pada Praktikum/modul/unguided.

Source code dan hasil eksekusi project modul-05-relasi-antar-kelas, package guided dan unguided.